

YogiTrack

Yoga H'om Studio Management System

Project Report — Application 2.0

ACS-5423-999 • Spring 2026

Des Dew

Contents

1. Project Overview	3
2. Deployment	3
3. Technology Stack	3
4. API Endpoints.....	4
Authentication	4
Customers	4
Instructors	4
Packages	4
Classes	5
Attendance.....	5
5. Use Cases Implemented	5
Use Case 1: Add/Modify/Delete Instructor.....	5
Use Case 2: Add/Modify/Delete Customer	5
Use Case 3: Add/Modify/Delete Package.....	6
Use Case 4: Add/Modify/Delete Class	6
Use Case 5: Record Class Attendance	6
6. Key Design Decisions.....	7
6.1 People Management View	7
6.2 Name-Check Gate Pattern	7
6.3 Collision-Safe ID Generation	7
6.4 Schedule Conflict Detection	7
6.5 Role-Based Authentication.....	7
6.6 Dual-Layer Validation.....	8
6.7 UI Design	8
7. Data Model.....	10
8. UML Models	11
8.1 Use Case Diagram.....	11
8.2 Entity Relationship Diagram (ERD)	12
8.3 Class Diagram	13
8.4 Sequence Diagram	14
Appendix: Generative AI Usage Disclosure	15
Tools Used.....	15

List of Figures

Figure 1 - Color-Coded Badges Example	8
Figure 2 - Unified People List Example.....	9
Figure 3 - Name-Check Gate Example	9
Figure 4 - Per-Column Filtering.....	9
Figure 5 - Role-Based Dashboard Examples.....	10
Figure 6 - Application 2.0 Use Case Diagram	11
Figure 7 - ERD Diagram	12
Figure 8 - Class Diagram.....	13
Figure 9 - Sequence Diagram for Record Class Attendance.....	14

1. Project Overview

Yoga H'om is a yoga studio which serves the surrounding suburbs of Pittsburgh, Pennsylvania. It offers yoga classes and workshops daily to people of all abilities. Currently, it handles its instructor information, customer registration, class package sales and student attendance through non-computerized methods (manual paper based). The need for an automated method became apparent as the number of students at Yoga H'om has increased. This automation will allow for improved operational efficiency and reduce the chance of recording errors.

YogiTrack is a web-based Studio Management Application created to streamline the main day-to-day functions of Yoga H'om. YogiTrack is focused on the initial stage of this project (Application 1.0) – automating Instructor/Customer Management while maintaining simplicity and ease of use throughout, allowing access from any modern internet browser.

Application 2.0 expands on the foundation of Application 1.0, adding Package management, Class scheduling with conflict detection, Class Attendance recording, and a full authentication system with role-based access control for Managers and Instructors.

2. Deployment

Item	Detail
Live Application	https://yogitrack-des-9bc253af7715.herokuapp.com/
GitHub Repository	https://github.com/desireeamber-sudo/yogitrack
Platform	Heroku (Eco Dyno)
Database	MongoDB Atlas (cloud-hosted)
CI/CD Pipeline	GitHub-connected auto-deploy — push to main branch triggers automatic Heroku deployment
Manager Login	manager@yogahom.com / Manager123!
Instructor Login	instructor@yogahom.com / Instructor123!

3. Technology Stack

YogiTrack is built on a modern JavaScript full-stack architecture, chosen for rapid development, cloud compatibility, and strong community support.

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript	UI pages, forms, dynamic table rendering, Fetch API for async calls
Backend	Node.js, Express.js	REST API, routing, business logic, session management
Database	MongoDB, Mongoose	NoSQL document storage, schema validation, enum enforcement
Authentication	express-session, bcryptjs	Session-based auth, password hashing, role-based access control
Deployment	Heroku	Cloud hosting, CI/CD pipeline, environment variable management
Version Control	Git, GitHub	Source control, CI/CD integration, commit history

4. API Endpoints

YogiTrack exposes a RESTful API with full CRUD operations for all resources. All endpoints except /auth/login require an active session.

Authentication

Method	Endpoint	Description
POST	/auth/login	Login with email and password
POST	/auth/logout	Destroy session and log out
POST	/auth/register	Register new user (Manager only)
GET	/auth/users	Get all users (Manager only)
DELETE	/auth/users/:id	Delete a user (Manager only)
GET	/auth/me	Get current session info

Customers

Method	Endpoint	Description
GET	/customers	Get all customers sorted by first name
GET	/customers/:id	Get one customer by MongoDB ID
POST	/customers	Create new customer (duplicate name check, ID generation)
PUT	/customers/:id	Update customer record
DELETE	/customers/:id	Delete customer record

Instructors

Method	Endpoint	Description
GET	/instructors	Get all instructors
GET	/instructors/:id	Get one instructor by MongoDB ID
POST	/instructors	Create new instructor (duplicate name check, ID generation)
PUT	/instructors/:id	Update instructor record
DELETE	/instructors/:id	Delete instructor record

Packages

Method	Endpoint	Description
GET	/packages	Get all packages sorted by name
GET	/packages/:id	Get one package by MongoDB ID
POST	/packages	Create new package
PUT	/packages/:id	Update package record
DELETE	/packages/:id	Delete package record

Classes

Method	Endpoint	Description
GET	/classes	Get all classes sorted by day and time
GET	/classes/:id	Get one class by MongoDB ID
POST	/classes	Create new class (schedule conflict check)
PUT	/classes/:id	Update class (conflict check excludes current record)
DELETE	/classes/:id	Delete class record

Attendance

Method	Endpoint	Description
GET	/attendance	Get all attendance records sorted by date
GET	/attendance/:id	Get one attendance record by MongoDB ID
POST	/attendance	Record attendance and deduct class balance for each customer
DELETE	/attendance/:id	Delete attendance record

5. Use Cases Implemented

Application 2.0 implements five fully deployed use cases. All use cases were verified locally and on Heroku.

Use Case 1: Add/Modify/Delete Instructor

Actor

Manager

Description

The manager creates, updates, or removes instructor records. Each record contains identifying and contact information.

Data Fields

- Instructor ID (auto-generated, prefix I, e.g. I00001)
- First Name, Last Name, Address, Phone, Email
- Preferred Mode of Communication (phone or email)

Basic Flow

1. Manager navigates to Add Instructor and enters first and last name.
2. YogiTrack checks for duplicate names. If found, manager confirms before proceeding.
3. Remaining fields are revealed. Manager completes and submits the form.
4. YogiTrack validates all fields, generates an Instructor ID, saves the record, and confirms.
5. Manager is redirected to the People List.

Use Case 2: Add/Modify/Delete Customer

Actor

Manager

Description

The manager creates, updates, or removes customer records. Each record includes contact information and a class balance.

Data Fields

- Customer ID (auto-generated, prefix C, e.g. C00001)
- First Name, Last Name, Address, Phone, Email

- Preferred Mode of Communication (phone or email)
- Class Balance (initialized to 0 on creation)

Basic Flow

6. Manager navigates to Add Customer and enters first and last name.
7. YogiTrack checks for duplicate names. If found, manager confirms before proceeding.
8. Remaining fields are revealed. Manager completes and submits the form.
9. YogiTrack validates all fields, generates a Customer ID with C prefix, sets class balance to 0, saves the record, and confirms.
10. Manager is redirected to the People List.

Use Case 3: Add/Modify/Delete Package

Actor

Manager

Description

The manager defines the class packages offered by the studio. Packages represent the products customers can purchase to attend classes.

Data Fields

- Package ID (auto-generated, prefix P, e.g. P00001)
- Package Name
- Category (General or Senior)
- Number of Classes (1, 4, 10, or Unlimited)
- Class Type (General or Special)
- Start Date, End Date
- Price

Basic Flow

11. Manager navigates to Add Package and completes all fields.
12. YogiTrack validates all required fields.
13. YogiTrack generates a Package ID, saves the record, and displays a confirmation.
14. Manager is redirected to the Package List.

Use Case 4: Add/Modify/Delete Class

Actor

Manager

Description

The manager schedules classes, assigning an instructor, day, time, class type, and pay rate. YogiTrack enforces that only one class can be held at the same day and time.

Data Fields

- Class ID (auto-generated, prefix CL, e.g. CL00001)
- Instructor ID (selected from existing instructors)
- Day (Sunday through Saturday)
- Time
- Class Type (General or Special)
- Pay Rate

Basic Flow

15. Manager navigates to Add Class and selects an instructor from the dropdown.
16. Manager selects day, time, class type, and pay rate.
17. YogiTrack checks for a scheduling conflict. If a class already exists at that day and time, an error message is returned and the manager must choose a different slot.
18. YogiTrack generates a Class ID, saves the record, and confirms.
19. Manager is redirected to the Class Schedule.

Use Case 5: Record Class Attendance

Actor

Instructor

Description

The instructor records which customers attended a class. YogiTrack deducts one class credit from each attending customer's balance and displays a confirmation message for each customer.

Basic Flow

20. Instructor navigates to Record Attendance and selects their instructor account.
21. YogiTrack displays only the classes assigned to that instructor.
22. Instructor selects a class. YogiTrack pre-fills the current date and time.
23. YogiTrack displays a schedule warning if the selected date/time does not match the class schedule.
24. Instructor selects the customers who attended from the customer list, which shows each customer's current class balance.
25. If any selected customer has a zero or negative balance, YogiTrack warns the instructor and asks whether to proceed with a negative balance to be resolved later.
26. On save, YogiTrack records the attendance and deducts one credit from each attending customer's balance.
27. YogiTrack displays a check-in confirmation message for each customer showing their updated balance.

6. Key Design Decisions

6.1 People Management View

Rather than maintaining separate customer and instructor list pages, Application 1.0 presents a unified People List that combines both record types into a single view. Each row is visually tagged with a color-coded type badge (pink for Customer, lavender for Instructor). This reduces navigation steps and gives the manager a complete picture of all studio members at a glance.

The People List supports per-column filtering on all fields, allowing the manager to quickly narrow results by type, preferred contact, name, or any other attribute without leaving the page.

6.2 Name-Check Gate Pattern

Add Customer and Add Instructor each use a two-stage process. First, you enter just the name and click the Check Name button. Then, if the name has already been entered, the rest of the information requested will appear. In this way, YogiTrack can confirm whether another person's name has been entered prior to requiring you to spend time entering additional details. If a duplicate is confirmed, a pop-up window will ask you if you want to continue with adding the new record or cancel it altogether, providing you with flexibility since it is possible that two individuals could have the same name.

6.3 Collision-Safe ID Generation

YogiTrack generates unique identifiers for all records by finding the current maximum identifier in MongoDB and incrementing from there, rather than counting documents. This approach is used across all six auto-generated ID types (C, I, P, CL, AT prefixes). Since identifiers are never reused after a record is deleted, a count-based approach would re-issue previously assigned IDs after deletions, this collision-safe method guarantees that no two records will ever share the same identifier.

6.4 Schedule Conflict Detection

The class controller checks for an existing class at the same day and time before saving. On updates, the current record is excluded from the conflict check so a class can be edited without falsely triggering a conflict against itself.

6.5 Role-Based Authentication

Authentication uses express-session for session management and bcryptjs for password hashing. Two roles are defined: Manager (full access to all features including user management) and Instructor (access limited to recording and viewing attendance and class schedule). Static HTML pages that require Manager access perform a session check on load and redirect to login if the role does not match.

6.6 Dual-Layer Validation

Required field validation is enforced at three layers: HTML5 native validation on the frontend, Express controller checks on the backend, and Mongoose schema enforcement at the database level. Each required field is validated at submission on the frontend, then again by the Express controller before anything is written to MongoDB. Finally, Mongoose schema validation enforces field requirements and uses enum restrictions limiting preferredContact to phone or email, classType to General or Special, and category to General or Senior ensuring that data integrity is maintained regardless of how the API is accessed.

6.7 UI Design

YogiTrack uses a soft pastel pink and lavender color scheme throughout, evoking the calm aesthetic of a yoga studio environment. All pages share a single stylesheet (styles.css) to maintain visual consistency across the entire application.

Color-Coded Badges The People List uses color-coded type badges to visually distinguish Customers (pink) from Instructors (lavender) at a glance. This same pattern is extended to the Package List, where both Category (General/Senior) and Class Type (General/Special) are color-coded using the same pink and lavender palette, and to the Class Schedule where Class Type badges provide instant visual context.

People List									
Home									
ID	Type	First Name	Last Name	Address	Email	Phone	Pref. Contact	Balance	Actions
Filter...	All	Filter...	p	Filter...	Filter...	Filter...	All	Filter...	
C00003	Customer	Maria	Petrocelli	78 Forbes Ave, Pittsburgh, PA 15213	maria.p@outlook.com	412-555-3044	email	0	Edit Delete
I00003	Instructor	Rachel	Stemple	204 Mount Lebanon Blvd, Pittsburgh, PA 15228	r.stemple@yogahom.com	412-555-0392	email	—	Edit Delete

Package List								
Home Add New Package								
Package ID	Package Name	Category	No. of Classes	Class Type	Start Date	End Date	Price	Actions
Filter...	Filter...	All	4	All	Filter...	Filter...	Filter...	
P00002	4 Class Pass (General)	General	4	General	12/31/2025	1/30/2026	\$70	Edit Delete
P00005	4 Class Pass (Senior)	Senior	4	General	12/31/2025	1/30/2026	\$60	Edit Delete

Class Schedule						
Home Add New Class						
Class ID	Instructor ID	Day	Time	Class Type	Pay Rate	Actions
Filter...	Filter...	Wednesday	Filter...	All	Filter...	
CL00004	I00001	Wednesday	4:45 PM	Special	\$55/hr	Edit Delete
CL00003	I00003	Wednesday	9:00 AM	General	\$45/hr	Edit Delete

Figure 1 - Color-Coded Badges Example

Unified People Management View Rather than maintaining separate list pages for customers and instructors, YogiTrack combines both record types into a single People List with per-column filtering on all fields. This reduces navigation steps and gives the manager a complete picture of all studio members at once.

People List
Home

ID	Type	First Name	Last Name	Address	Email	Phone	Pref. Contact	Balance	Actions
Filter...	All	Filter...	Filter...	Filter...	Filter...	Filter...	All	Filter...	
C00001	Customer	Sandra	Dubowski	55 Mount Lebanon Blvd, Pittsburgh, PA 15228	sandra.dubowski@gmail.com	412-555-1021	email	4	Edit Delete
C00002	Customer	James	Harrington	301 Beechwood Blvd, Pittsburgh, PA 15217	j.harrington@gmail.com	412-555-2033	phone	10	Edit Delete
C00003	Customer	Maria	Petrocelli	78 Forbes Ave, Pittsburgh, PA 15213	maria.p@outlook.com	412-555-3044	email	0	Edit Delete
C00004	Customer	Dorothy	Blackwell	19 Frick Park Lane, Pittsburgh, PA 15217	d.blackwell@gmail.com	412-555-4055	email	2	Edit Delete
C00005	Customer	Anthony	Russo	543 Penn Ave, Pittsburgh, PA 15222	a.russo@yahoo.com	412-555-5066	phone	8	Edit Delete
C00006	Customer	Linda	Mazurek	112 Walnut St, Pittsburgh, PA 15232	linda.m@gmail.com	412-555-6077	email	1	Edit Delete
C00007	Customer	Robert	Gaffney	667 Murray Ave, Pittsburgh, PA 15217	r.gaffney@comcast.net	412-555-7088	phone	3	Edit Delete
I00001	Instructor	Margaret	Kowalski	412 Shadyside Ave, Pittsburgh, PA 15232	m.kowalski@yogahom.com	412-555-0181	email	—	Edit Delete
I00002	Instructor	Patricia	Novak	88 Squirrel Hill Rd, Pittsburgh, PA 15217	p.novak@yogahom.com	412-555-0247	phone	—	Edit Delete
I00003	Instructor	Rachel	Stemple	204 Mount Lebanon Blvd, Pittsburgh, PA 15228	r.stemple@yogahom.com	412-555-0392	email	—	Edit Delete

Figure 2 - Unified People List Example

Name-Check Gate Pattern The Add Customer and Add Instructor forms use a two-step process; the manager first enters only the name and clicks Check Name, and YogiTrack checks for duplicates before revealing the remaining fields. This prevents wasted effort filling out a full form before discovering a potential duplicate exists.

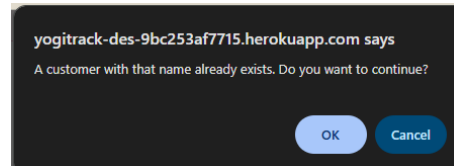


Figure 3 - Name-Check Gate Example

Per-Column Filtering All list pages — People, Packages, Classes, and Attendance Records; support per-column filtering that operates client-side with no page reloads. Dropdown filters are provided for enum fields and text filters for free-form fields, allowing the manager to narrow results instantly across any combination of columns.

People List
Home

ID	Type	First Name	Last Name	Address	Email	Phone	Pref. Contact	Balance	Actions
Filter...	All	Filter...	p	Filter...	Filter...	Filter...	All	Filter...	

Package List
Home Add New Package

Package ID	Package Name	Category	No. of Classes	Class Type	Start Date	End Date	Price	Actions
Filter...	Filter...	All	4	All	Filter...	Filter...	Filter...	

Class Schedule
Home Add New Class

Class ID	Instructor ID	Day	Time	Class Type	Pay Rate	Actions
Filter...	Filter...	Wednesday	Filter...	All	Filter...	

Figure 4 - Per-Column Filtering

Role-Based Dashboards Managers and Instructors see different home dashboards reflecting their permissions. The Manager dashboard exposes all use cases including User Management. The Instructor dashboard shows only the three functions available to their role - Record Attendance, View Attendance, and View Class Schedule.

Manager Dashboard Instructor Dashboard

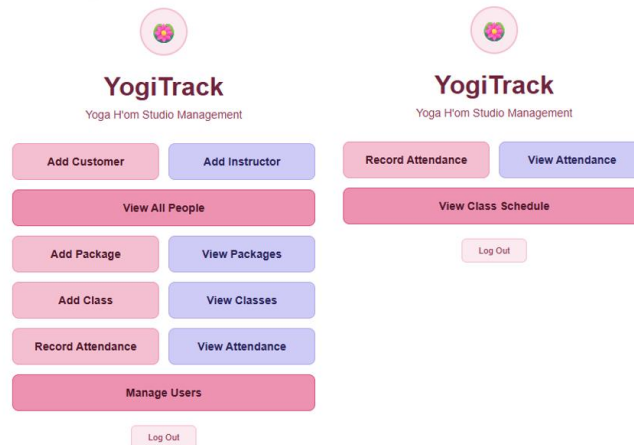


Figure 5 - Role-Based Dashboard Examples

7. Data Model

YogiTrack uses six MongoDB collections managed through Mongoose schemas.

Collection	ID Prefix	Key Fields	Notes
User	none (MongoDB _id)	email, role	Passwords stored as bcrypt hash. Role enum: Manager, Instructor
Customer	C (e.g. C00001)	customerId, classBalance	classBalance defaults to 0. Deducted on attendance.
Instructor	I (e.g. I00001)	instructorId	Linked to Class via instructorId field.
Package	P (e.g. P00001)	packageId, category, price	Standalone catalog. Not yet linked to Customer purchases.
Class	CL (e.g. CL00001)	classId, instructorId	instructorId references Instructor. Day/time conflict enforced.
Attendance	AT (e.g. AT00001)	classId, customers[]	classId references Class. customers is array of customerIds.

8. UML Models

8.1 Use Case Diagram

The use case diagram shows all actors and use cases. Five use cases are fully implemented in Application 2.0. Remaining use cases (Record a Sale, Generate Studio Reports, Publish Class Schedule, View Class Schedule, Generate Self-Performance Report) are planned for future phases.

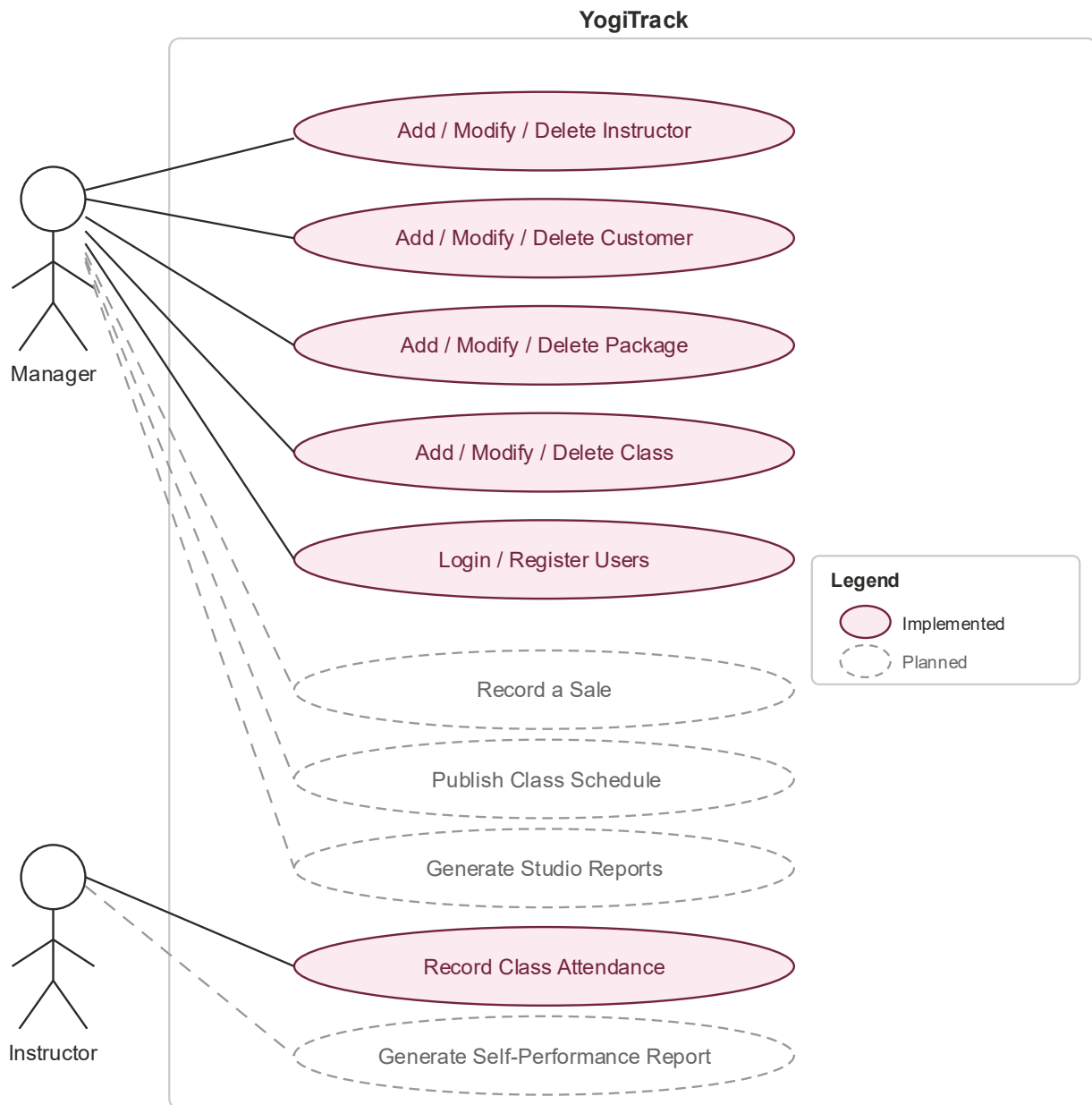


Figure 6 - Application 2.0 Use Case Diagram

8.2 Entity Relationship Diagram (ERD)

The ERD shows the six MongoDB collections and their relationships:

- Instructor leads Class (one Instructor to many Classes)
- Class is recorded in Attendance (one Class to many Attendance records)
- Customer attends Attendance (many Customers to many Attendance records via the customers array)
- User, Package are standalone collections with no foreign key relationships in the current implementation

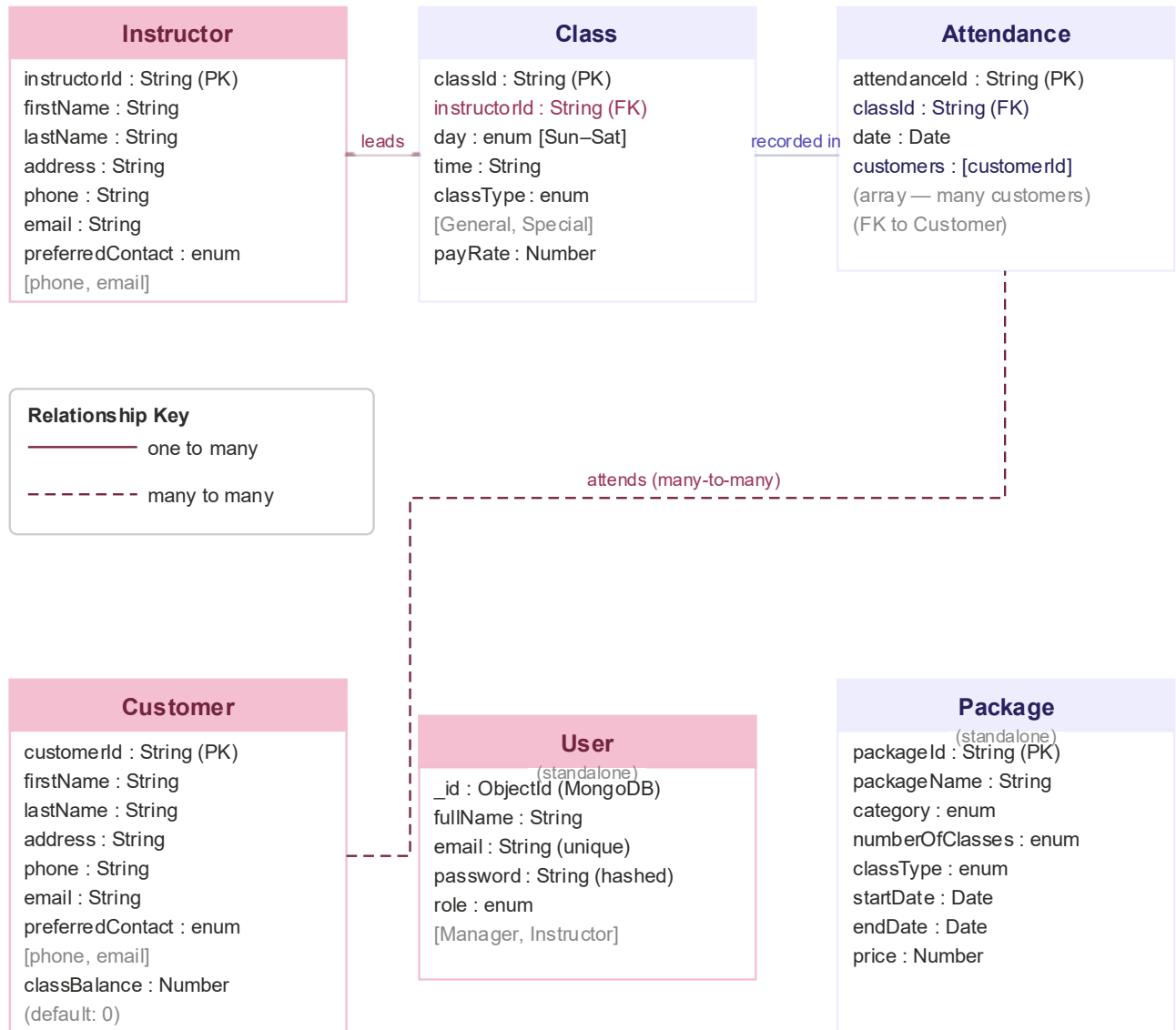


Figure 7 - ERD Diagram

8.3 Class Diagram

The class diagram shows the six Mongoose schema definitions with their fields and data types. All schemas enforce required fields and use enum validation where applicable.

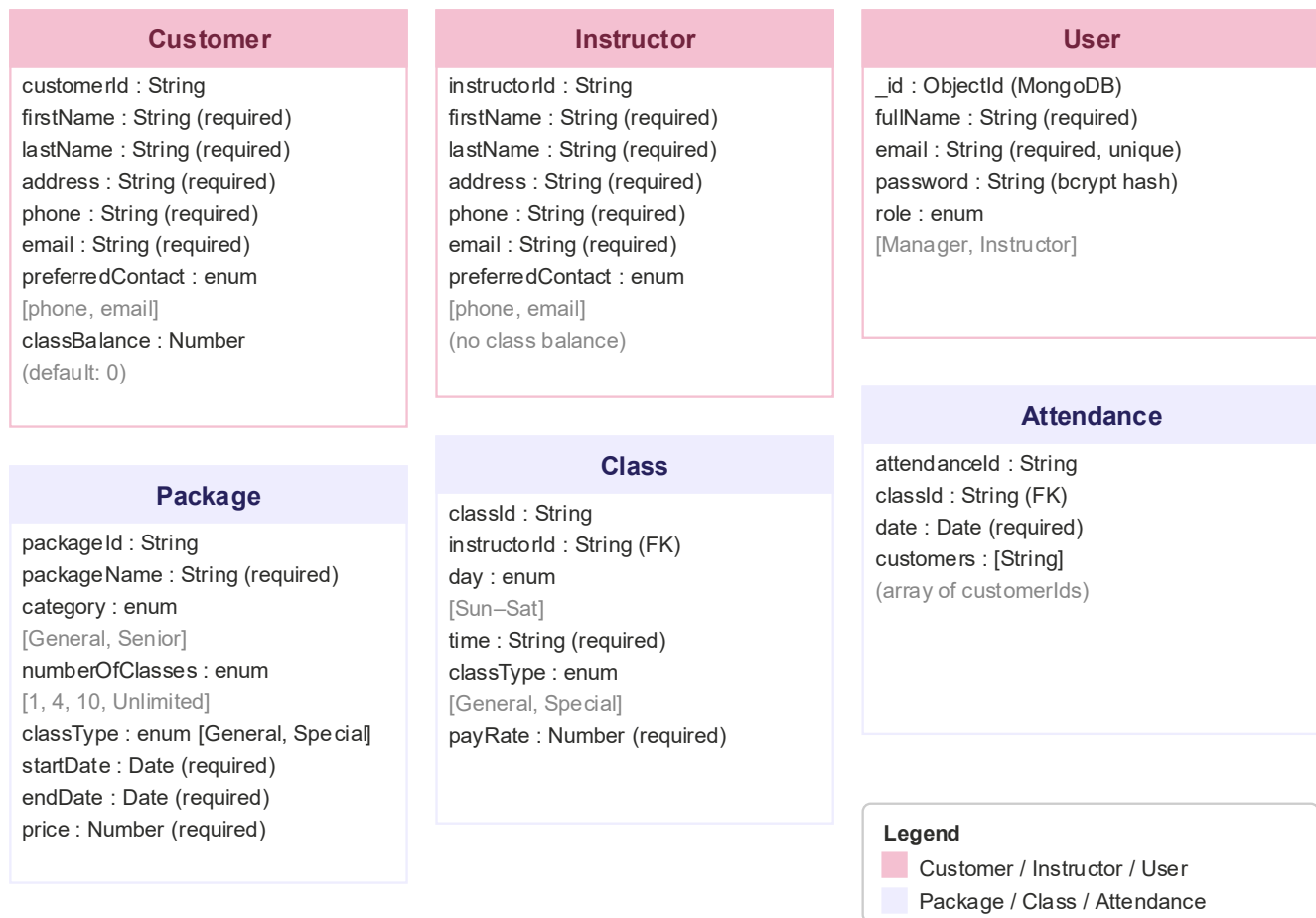


Figure 8 - Class Diagram

8.4 Sequence Diagram

The sequence diagram for Record Class Attendance shows the interaction between the Instructor (browser), Express API, and MongoDB: the instructor selects a class, the system loads customers with balances, the instructor selects attendees, the API validates balances, records attendance, and decrements each customer's classBalance in the database.

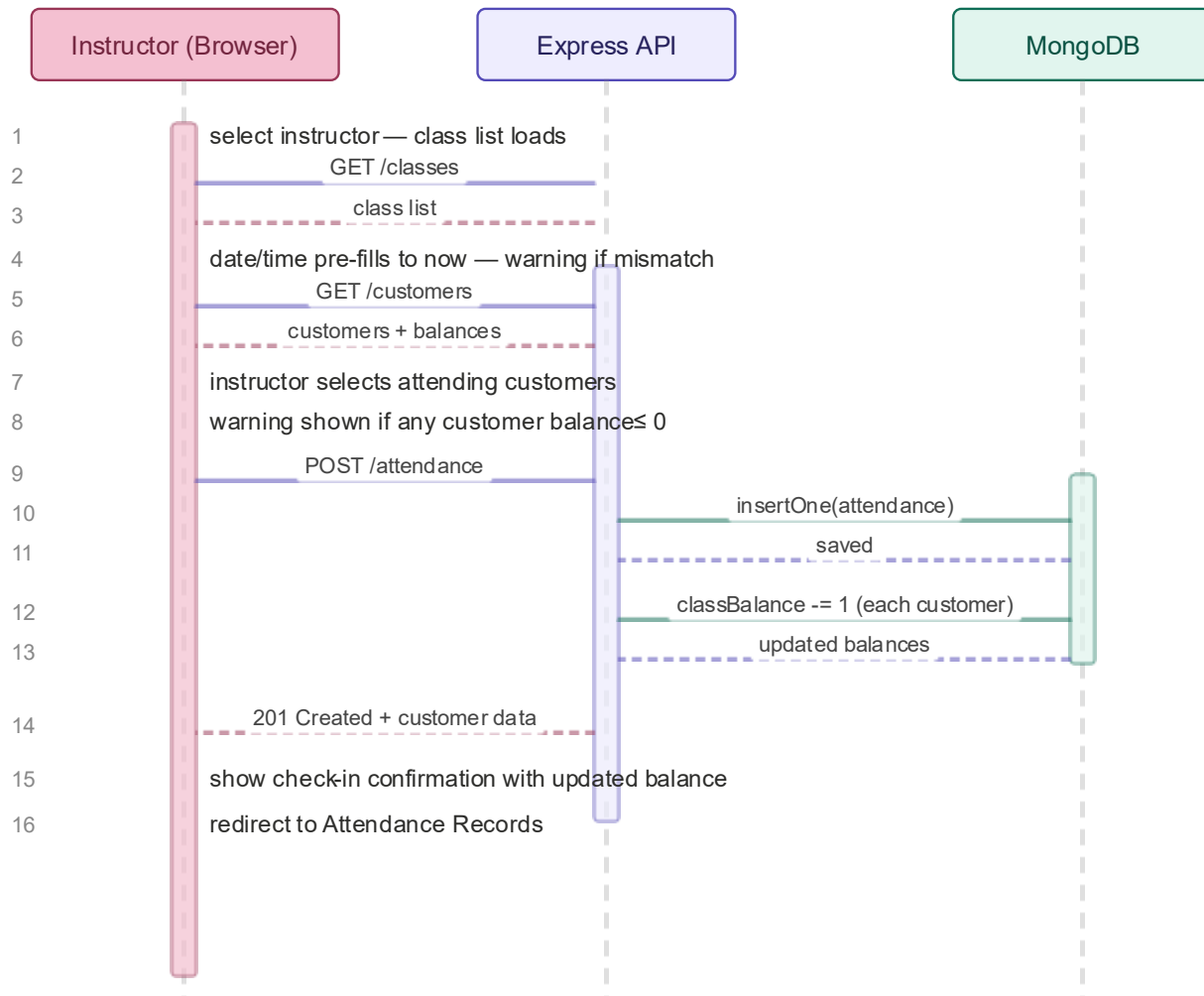


Figure 9 - Sequence Diagram for Record Class Attendance

Appendix: Generative AI Usage Disclosure

In accordance with the course AI policy, I am disclosing my use of generative AI tools throughout this project.

Tools Used

ChatGPT (OpenAI) was used during the initial project setup phase, including configuring the development environment, setting up Git and GitHub, connecting to MongoDB Atlas, and establishing the basic Express server structure.

Claude (Anthropic) was used throughout the development phase after ChatGPT became unresponsive mid-session. Claude assisted with the following:

- Debugging — identifying and fixing issues including non-responsive Edit and Delete buttons caused by DOM rebuilding in a loop, Customer ID collision bugs after record deletions, a header validation issue with the duplicate name check, and a pagination limit hiding records on the People List
- Code review — reviewing controller logic, API response structure, session middleware ordering, and frontend form behavior
- Planning and design — thinking through the unified People List approach, the two-step name-check form pattern, and the collision-safe ID generation strategy
- UI design — developing the pastel color palette, table layout, column filter functionality, and login page design
- Documentation — drafting the project report, README, seed scripts, and code comments

All code was tested locally and deployed to Heroku. I reviewed, tested, and can explain the function and logic of every component of the submitted application.